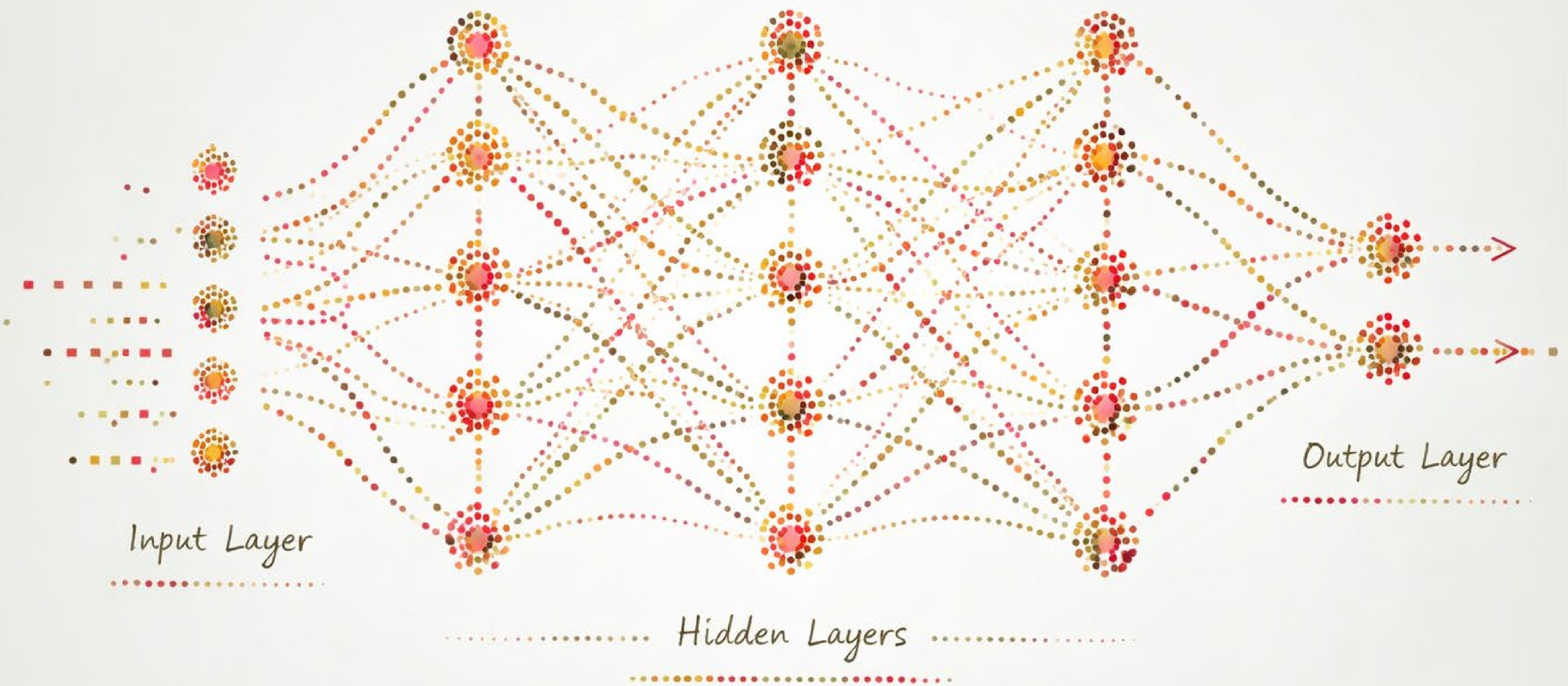
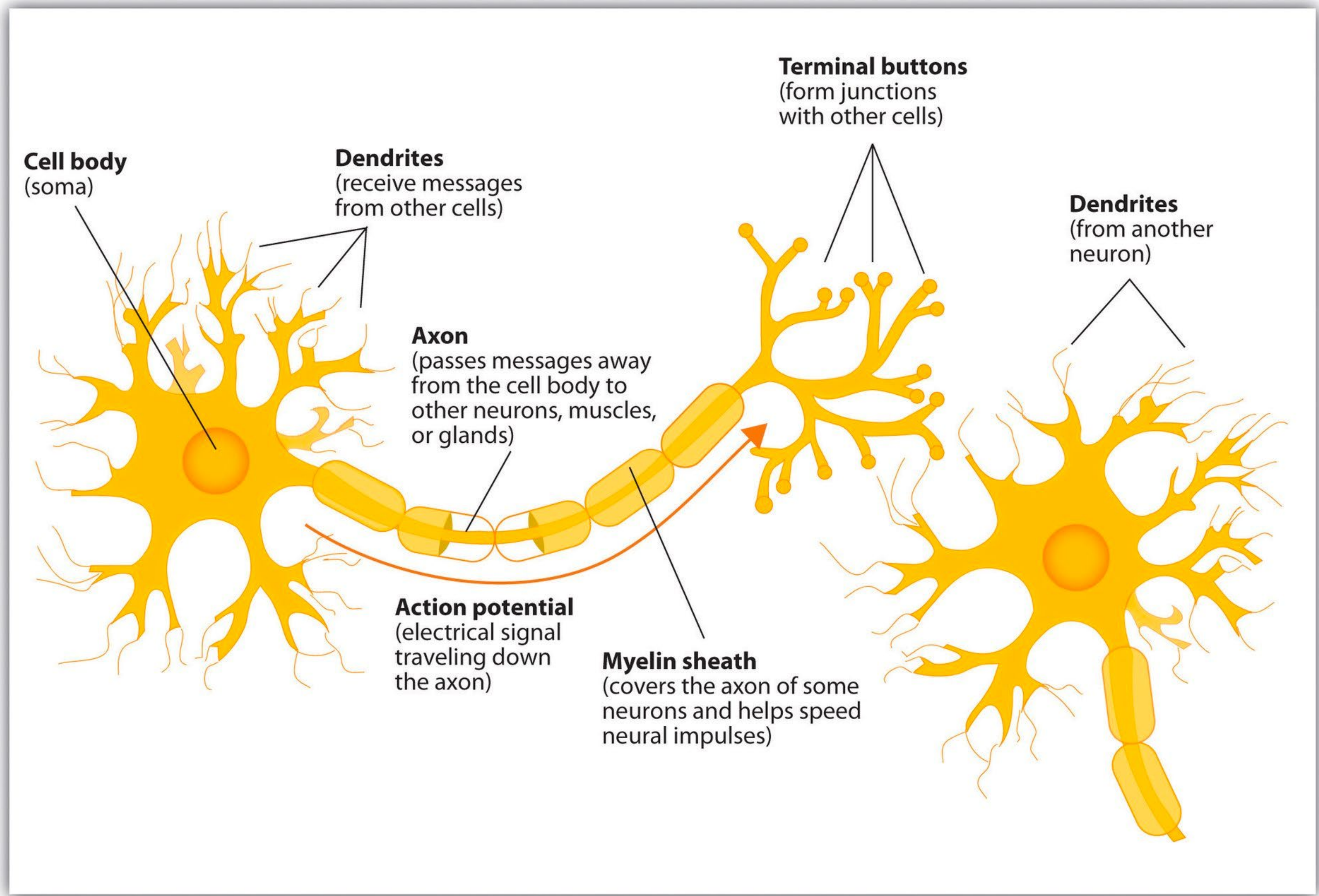


ARTIFICIAL NEURAL NETWORKS

ARTIFICIAL NEURAL NETWORKS

- Artificial neural networks (ANNs) are a class of machine learning models inspired by the structure and function of the brain
- The first ANN was introduced in 1943 by Warren McCulloch (a neurophysiologist) and Walter Pitts (a mathematician and logician)
- In the 1940s it was known that the brain is built of neurons wired together
  - Each neuron has inputs (dendrites)
  - As well as outputs (axon)
  - When impulses coming in exceed a threshold, the neuron emits a pulse, which is taken as input to the next neuron

BIOLOGICAL NEURON

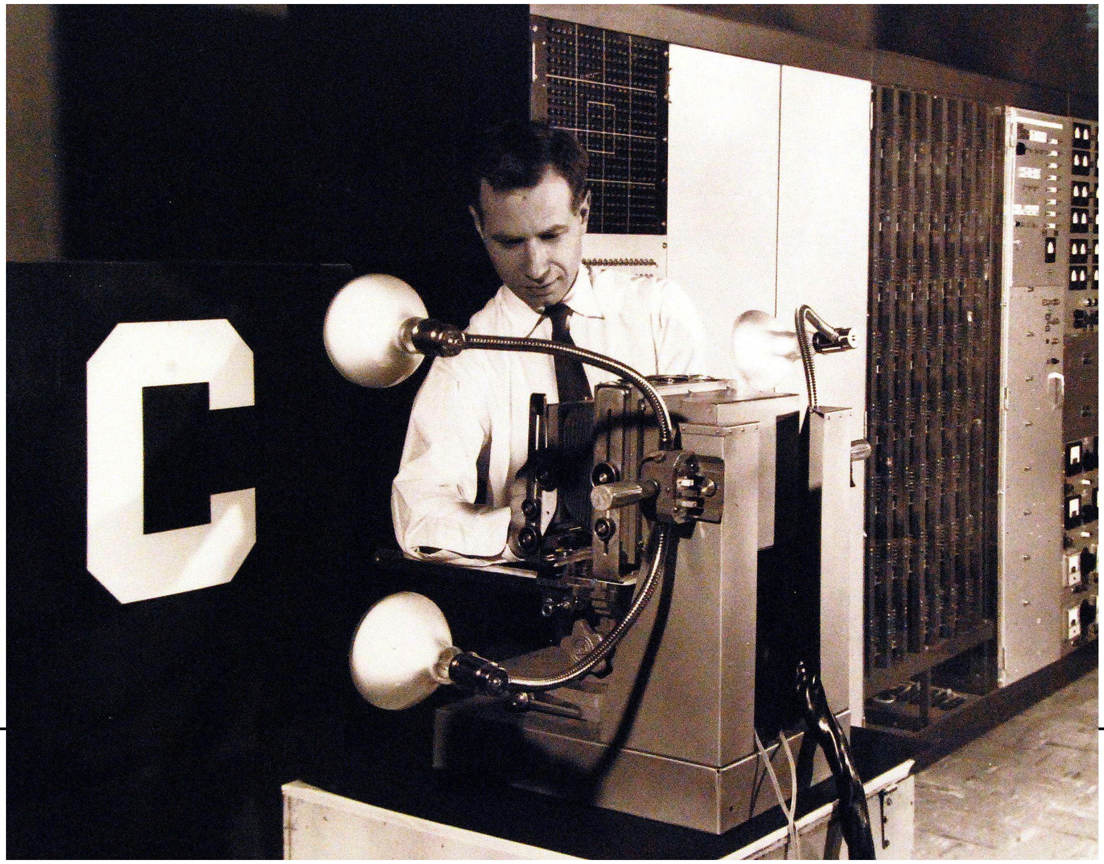


THE FIRST ANNS

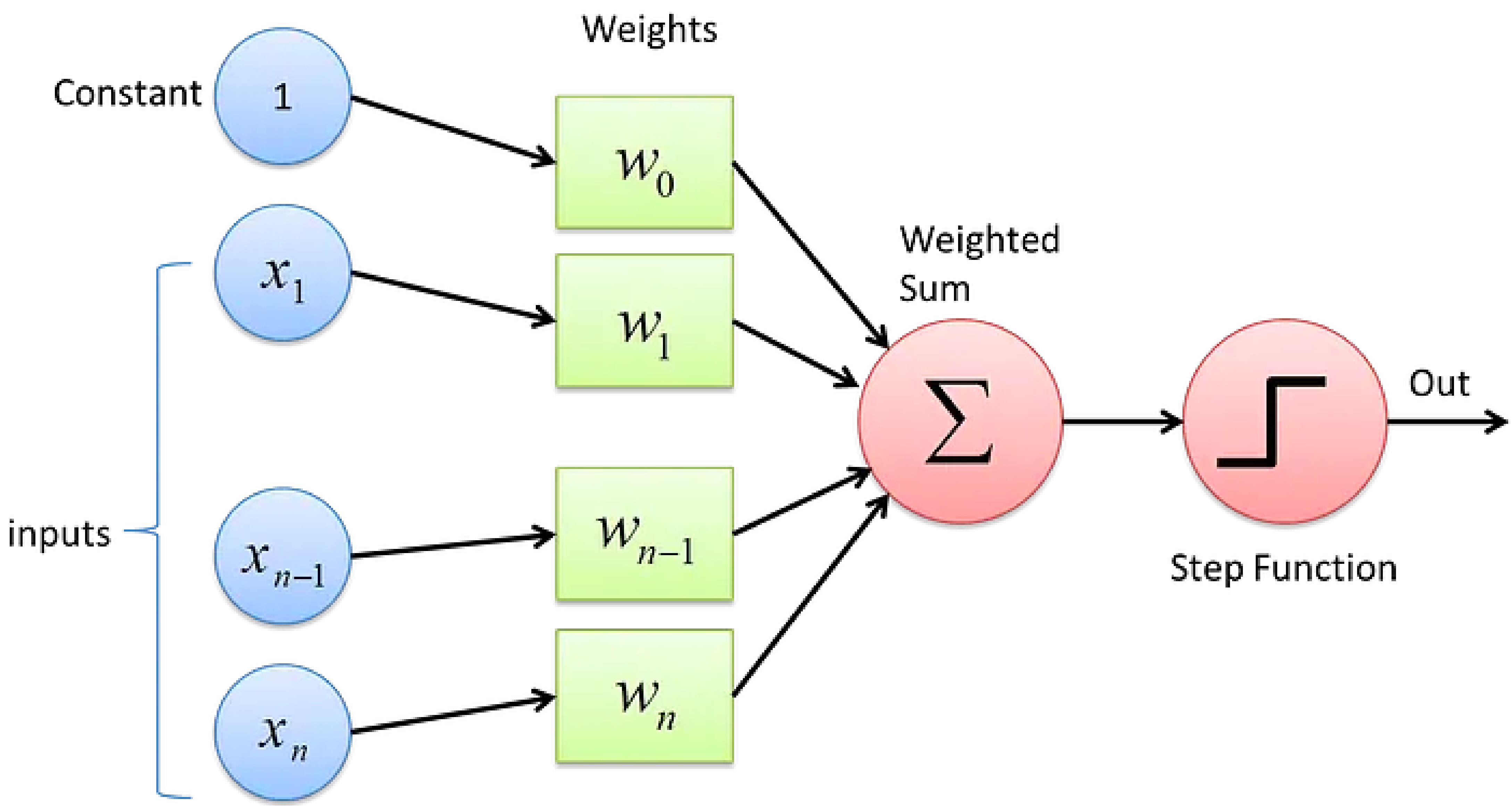
- The network proposed by McCulloch and Pitts was a binary threshold network
  - Inputs can only be binary
  - Used for simple logical operations
- In 1958, Frank Rosenblatt proposed a more complex ANN called the **Perceptron**
  - Inputs are weighted sums of inputs
  - Binary output is based on an activation function
  - Has the ability to <u>learn</u> by adjusting weights according to its mistakes

THE PERCEPTRON

- The name “perceptron” is in reference to a device that “perceives” a pattern
- Originally, it was intended to be a machine
- Built for binary classification in image recognition



THE PERCEPTRON



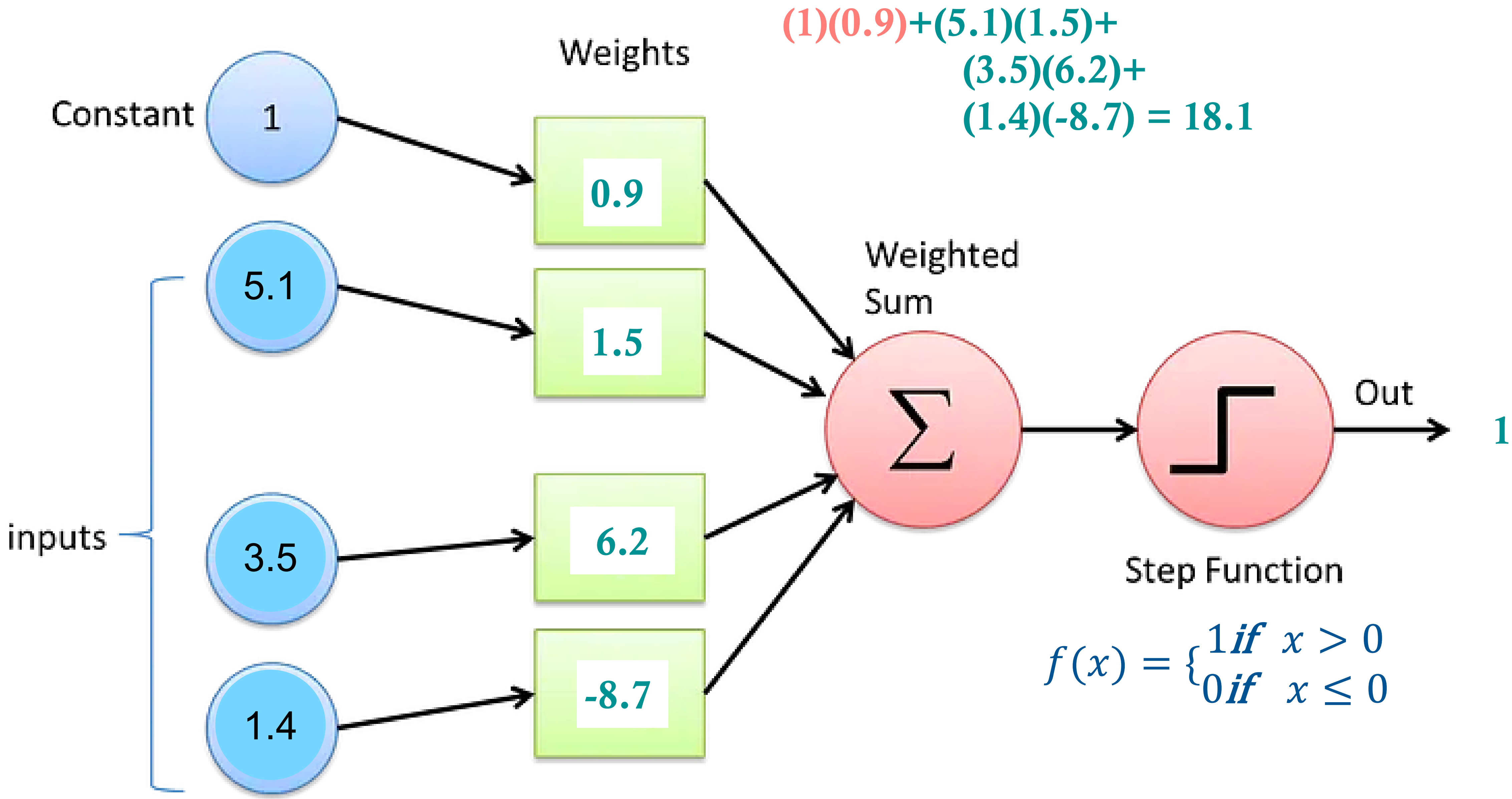
(1) Receive input

(2) Assign weights to the inputs

(3) Compute the weighted sum

(4) Generate output according to the activation function

THE PERCEPTRON



(1) Receive input

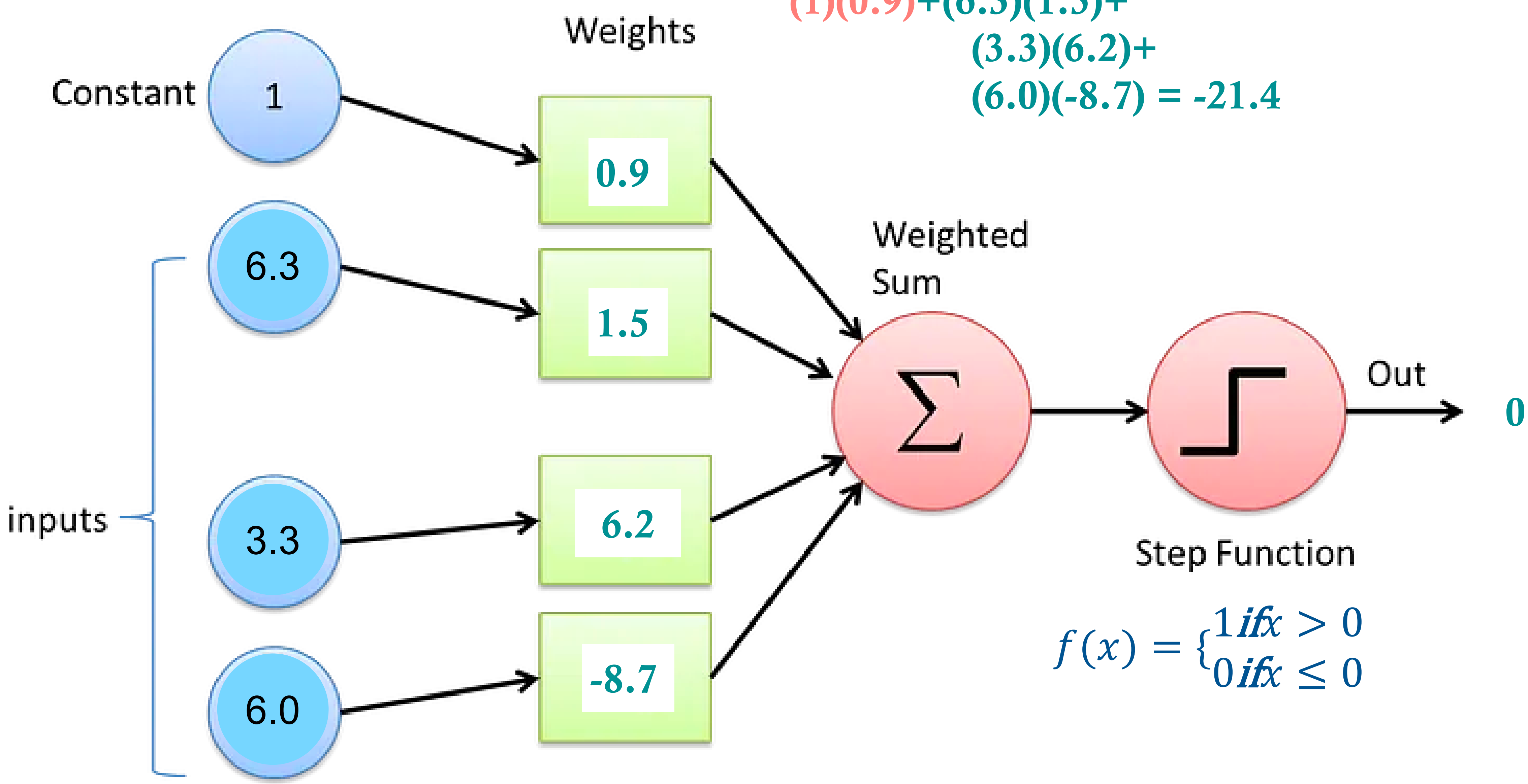
(2) Assign weights to the inputs

(3) Compute the weighted sum

(4) Generate output according to the activation function

THE PERCEPTRON

$$(1)(0.9)+(6.3)(1.5)+(3.3)(6.2)+(6.0)(-8.7) = -21.4$$



$$f(x) = \begin{cases} 1 \; if\, x > 0 \\ 0 \; if\, x \leq 0 \end{cases}$$

(1) Receive input

(2) Assign weights to the inputs

(3) Compute the weighted sum

(4) Generate output according to the activation function

TRAINING THE PERCEPTRON

- Training is the process of find the weights of the Perceptron
- Perceptron Algorithm:
  - Assign weights randomly
  - Compute the output
  - Compare the output to the true values
  - Adjust weights according to the error by:
    - increasing weights that were too low and
    - decreasing weights that were too high
  - Repeat

PERCEPTRON TRAINING ALGORITHM

1. Initialize the weights (and bias) with small random values or to zero
2. For each epoch:
   a. For each training instance:
      i. Compute the weighted sum: $z = w \cdot x + b$
      ii. Apply the activation function: $\hat{y} = \begin{cases} 1, & if \;\; z > 0 \\ 0, & otherwise \end{cases}$

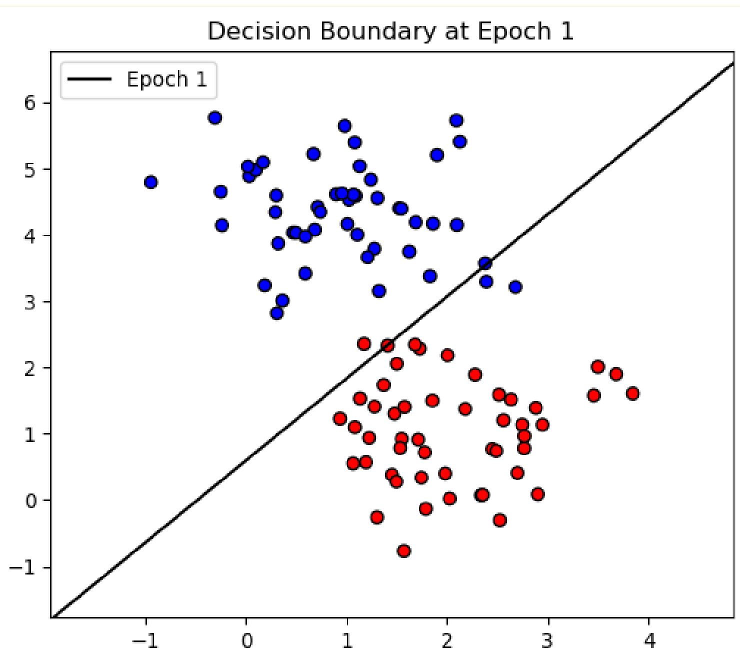
      .iii Update the weights if misclassified:

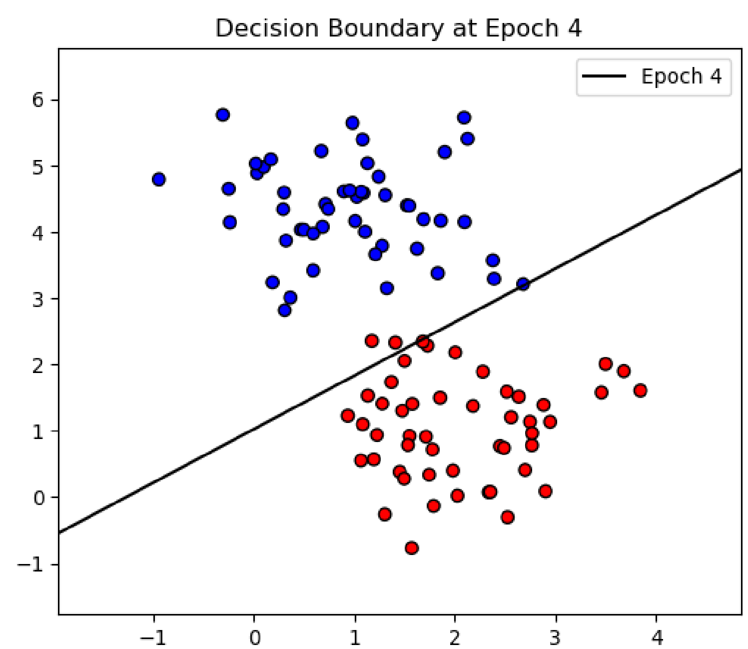
$$w \leftarrow w + \eta \, (y - \hat{y}) \, x$$

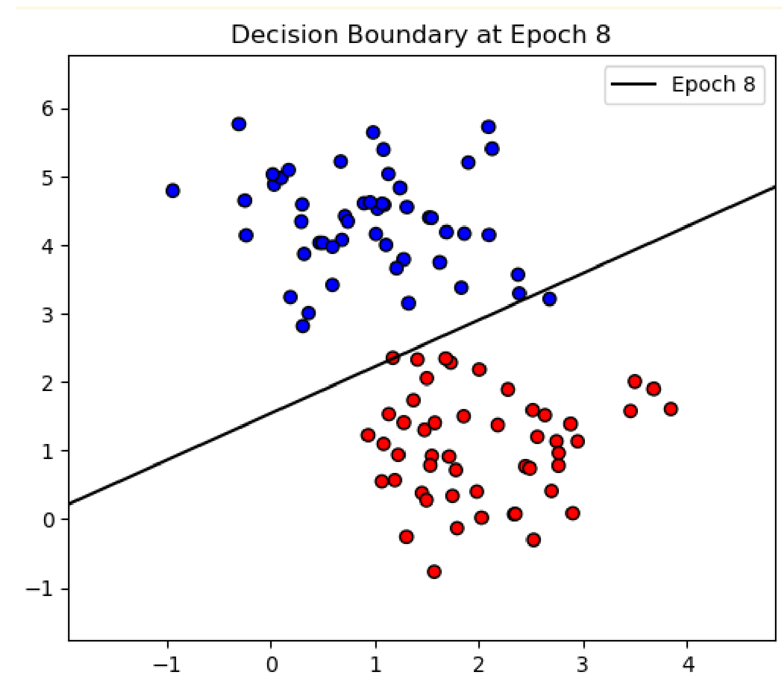
$$b \leftarrow b + \eta \, (y - \hat{y})$$

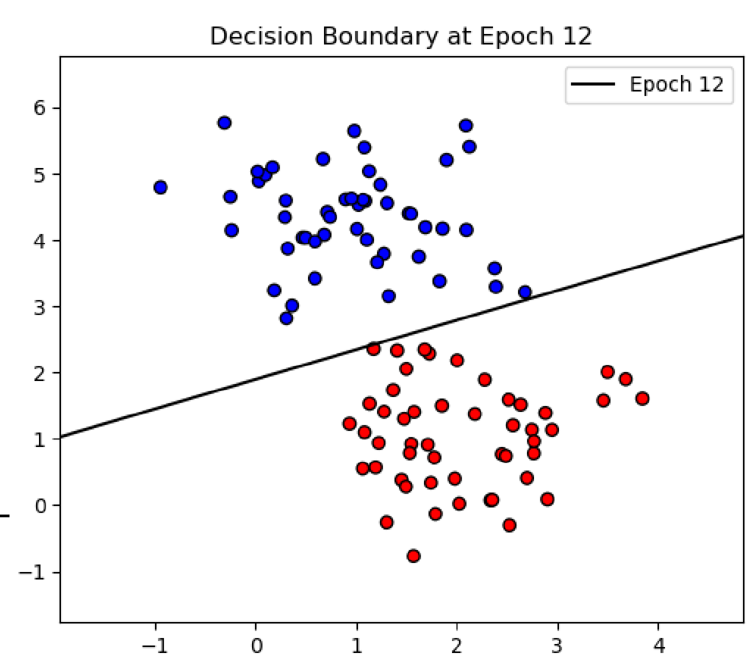
PERCEPTRON TRAINING ALGORITHM

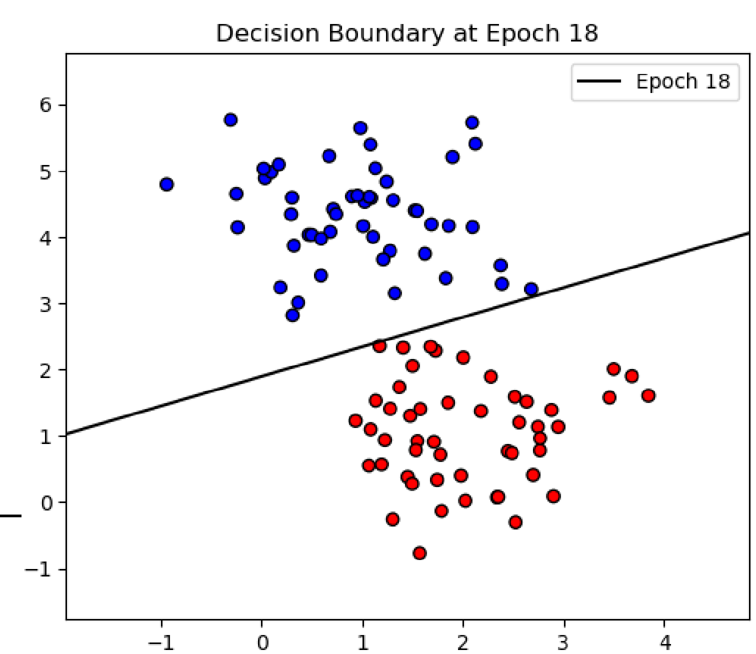
- If the classes are linearly separable, the perceptron algorithm will converge







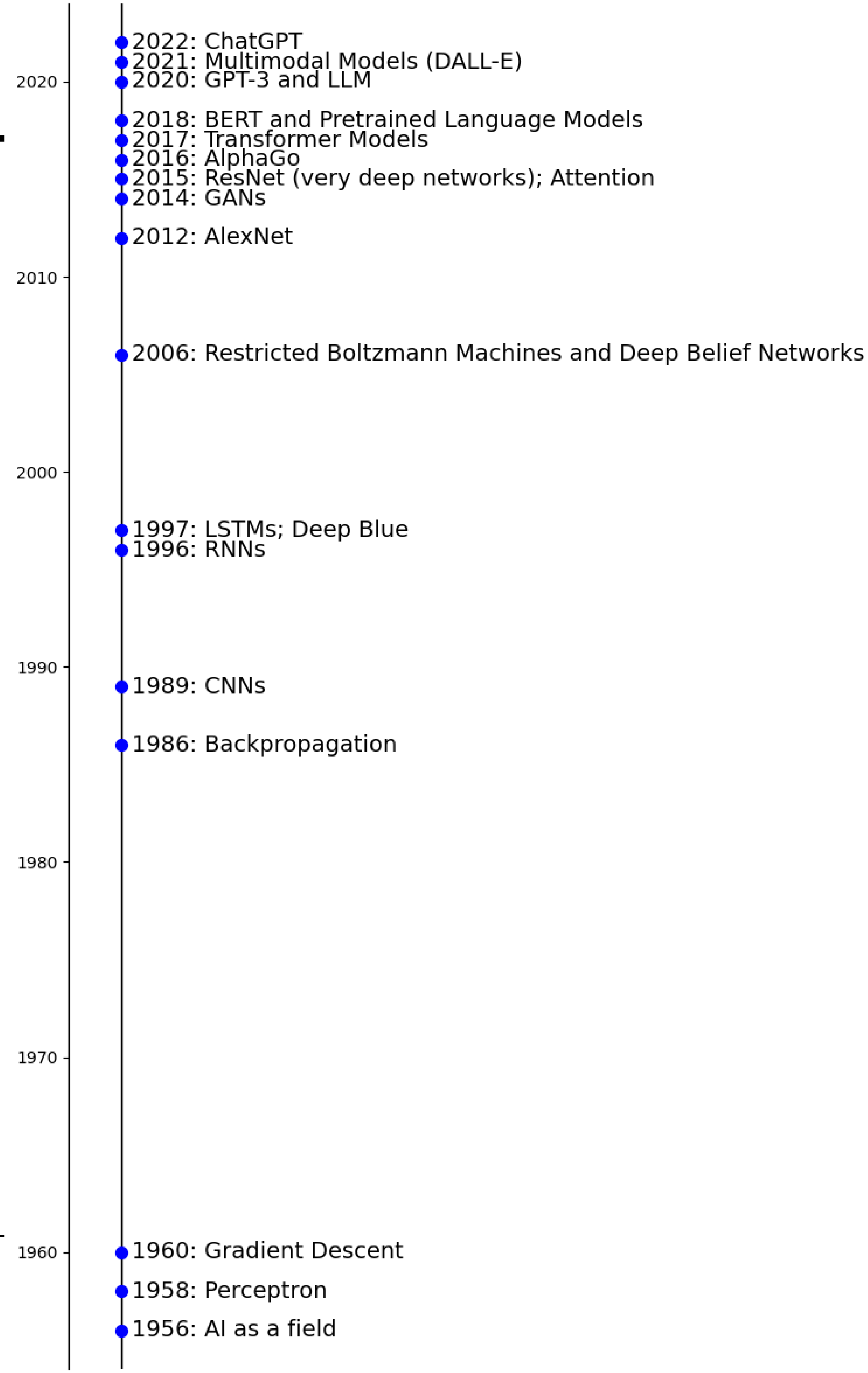




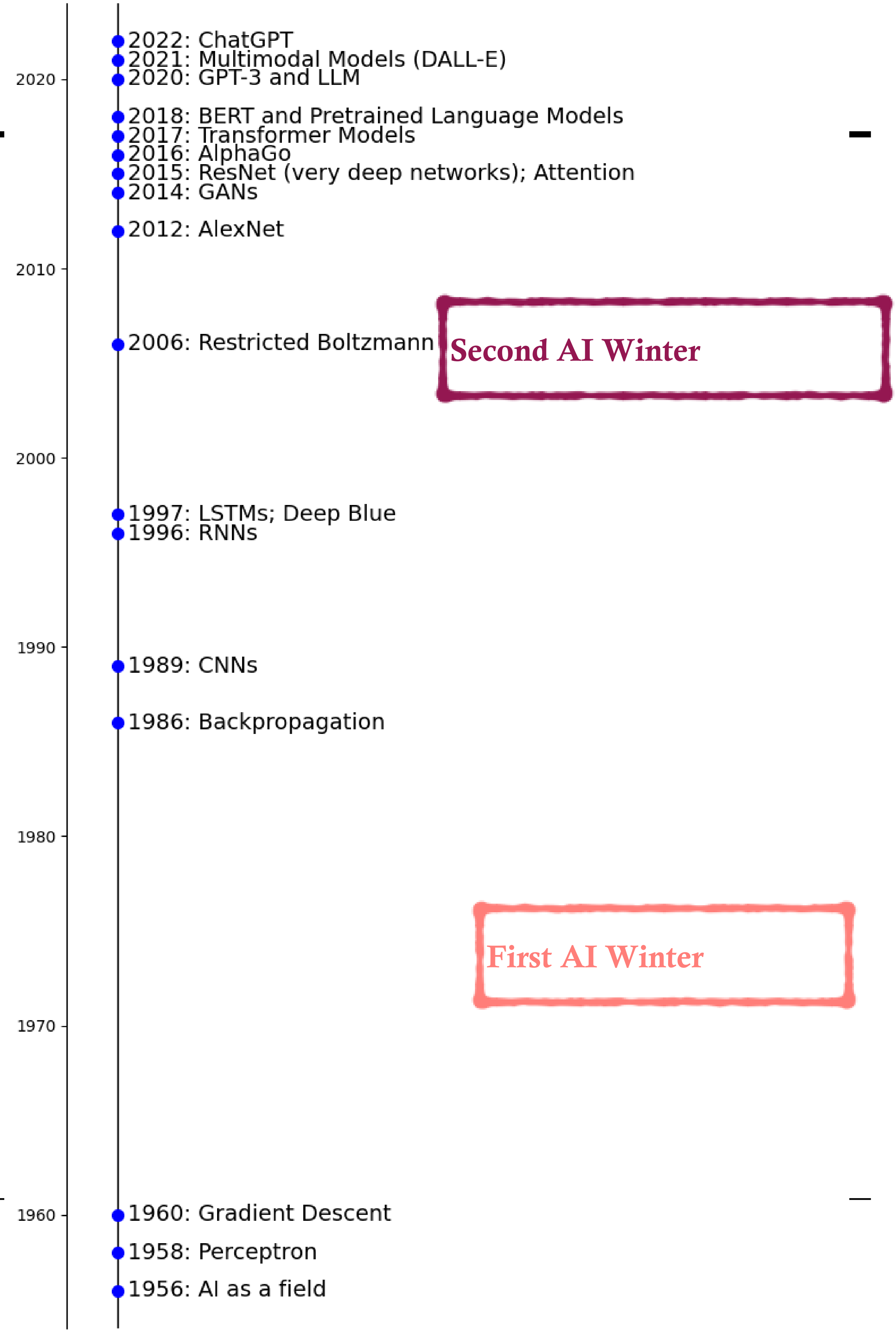
FIRST AI WINTER

- Originally there was a lot of excitement about ANNs
- In 1969, a book by Minsky and Papert highlighted several weaknesses of the Perceptron
  - The book demonstrates that there several basic patterns that the Perception would never be able to recognize
- Research and more importantly funding for research in ANN is essentially stopped for over 10 years

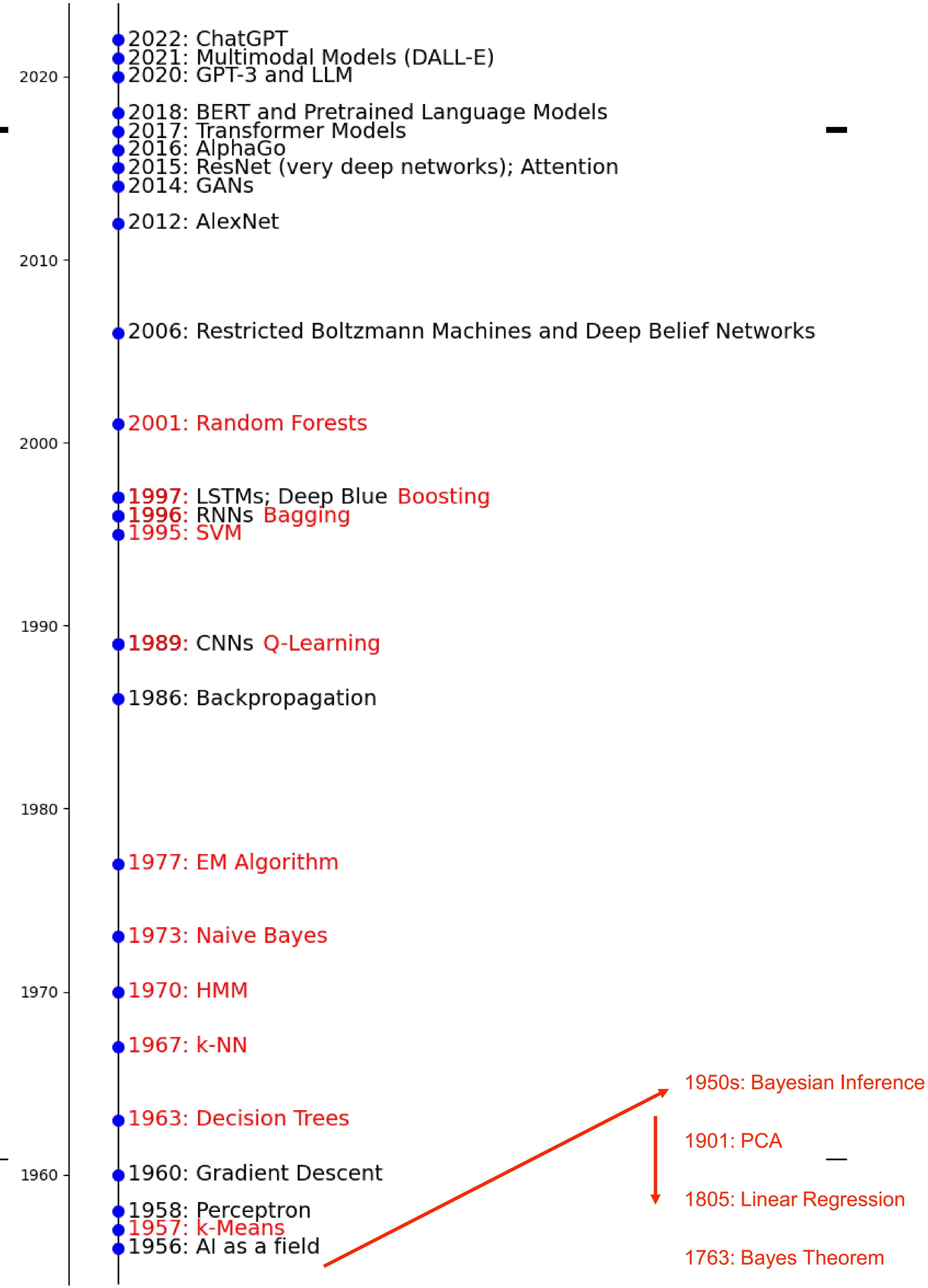
TIMELINE OF DEEP LEARNING MILESTONES



TIMELINE OF DEEP LEARNING MILESTONES



TIMELINE OF DEEP LEARNING MILESTONES



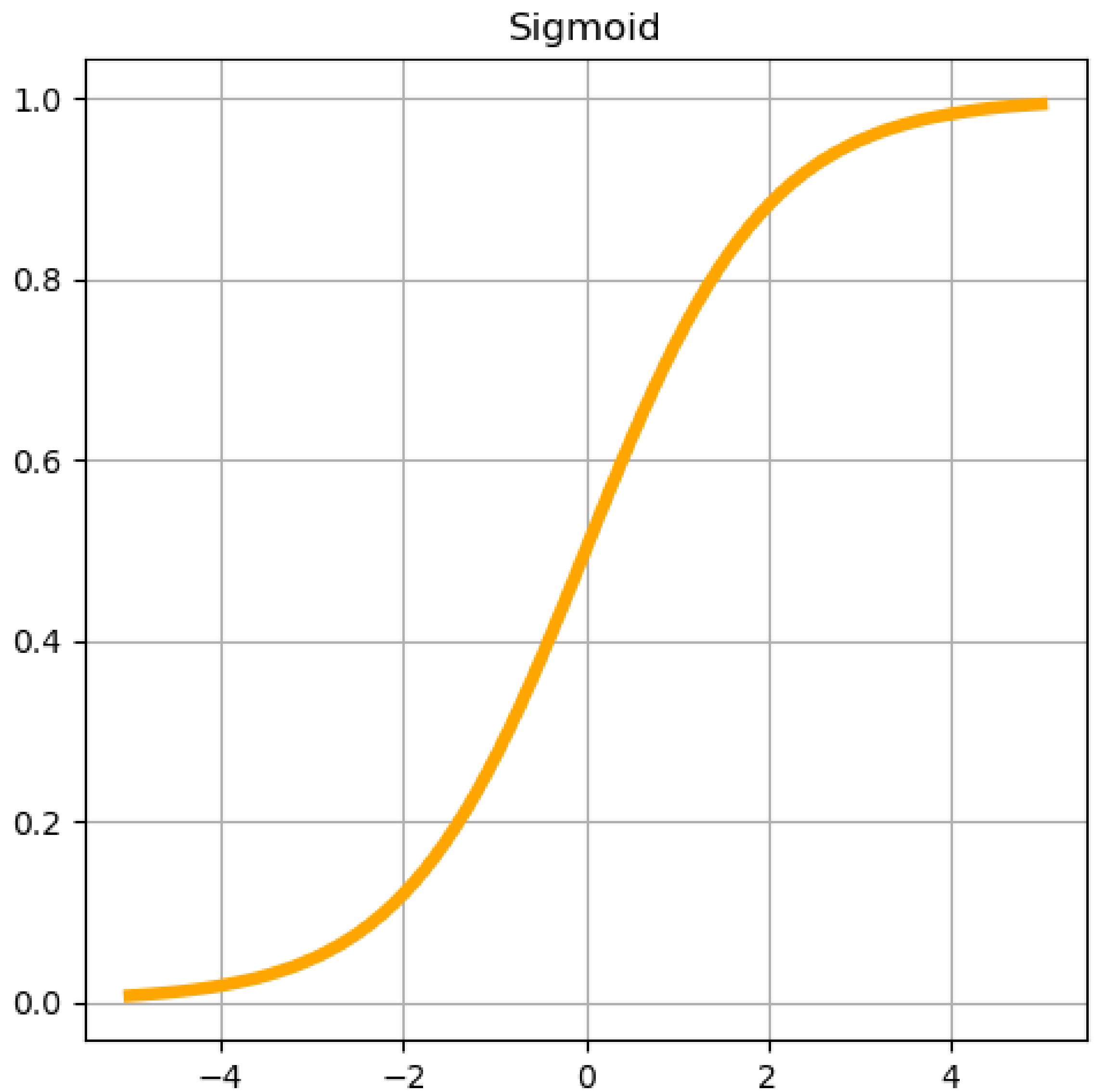
RELEVANCE OF THE PERCEPTRON

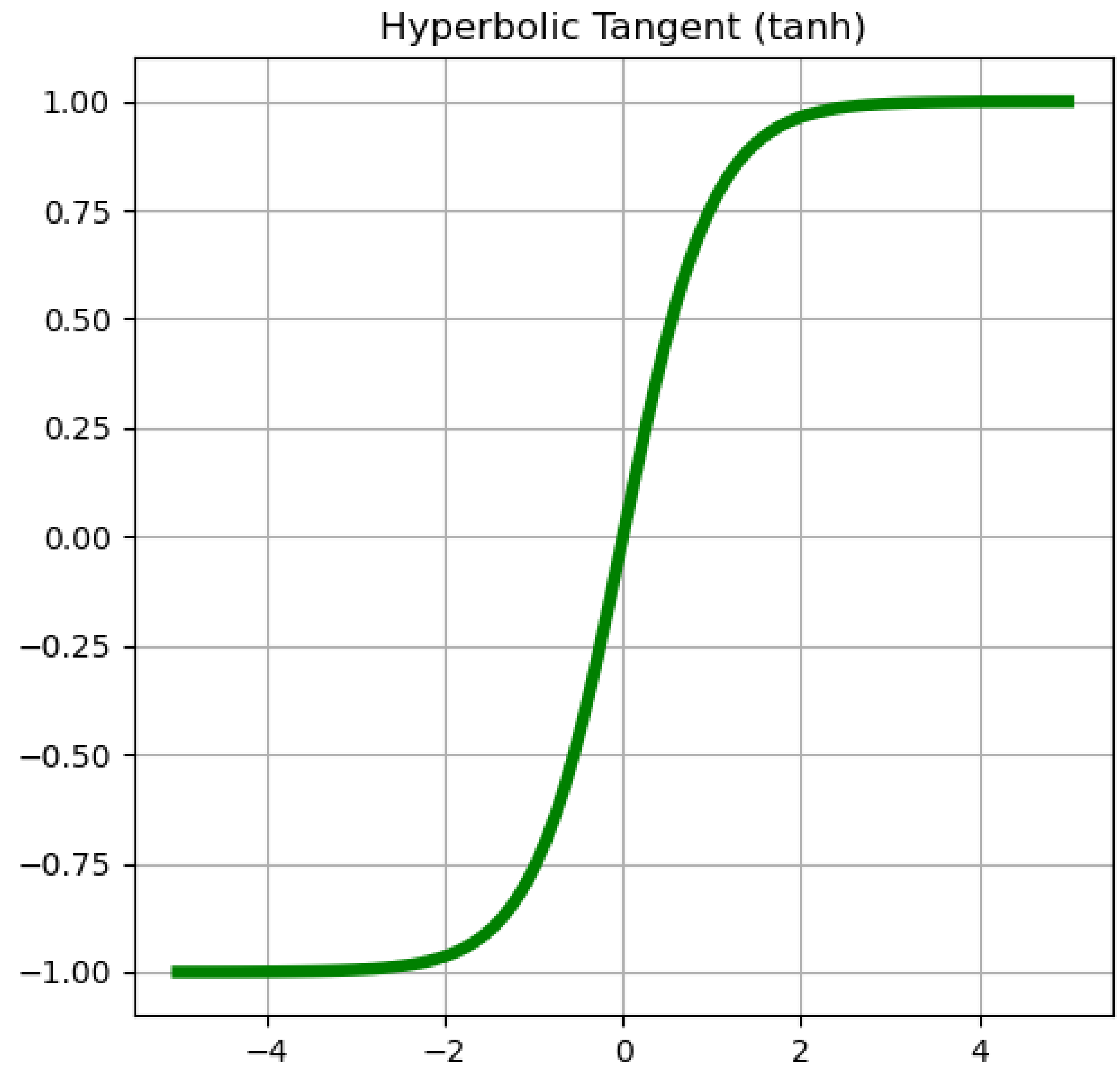
- Birth of Neural Networks
  - Early example of learning with data
  - Introduced concepts:
    - Weights and biases (compared to neurons in brain)
    - Updating based on mistakes
- Inspired Multilayer Perceptron and later advances in deep learning

IMPROVEMENTS TO THE PERCEPTRON

- Nonlinear activation functions
  - The original step function activation is not able to capture complex nonlinear relationships
  - Nonlinear differentiable functions (such as the sigmoid or hyperbolic tangent were introduced
- Multiple layers
  - Hidden layers can capture complex, nonlinear relationships
  - Each layer can transform the input in sophisticated ways

NONLINEAR ACTIVATION FUNCTIONS



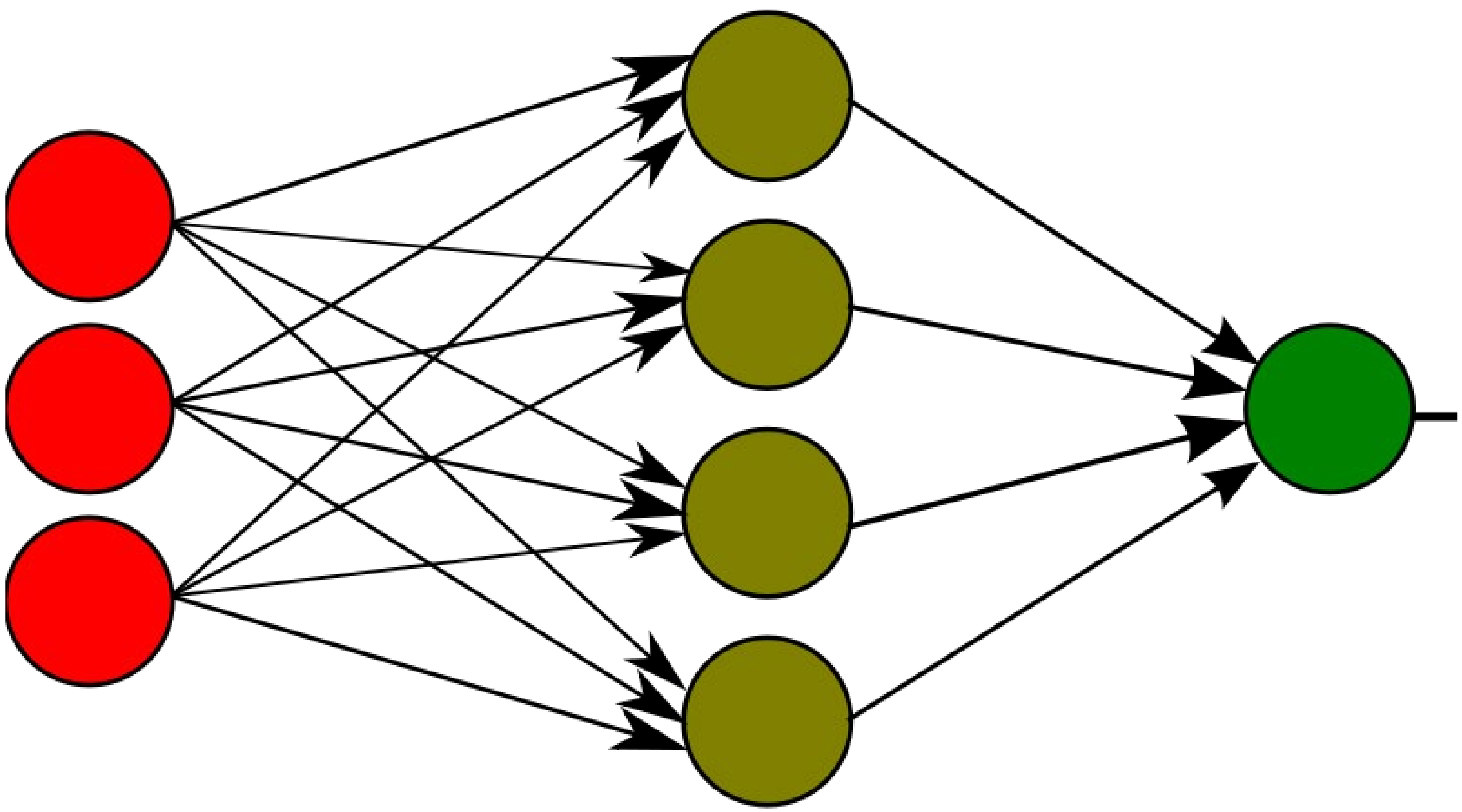


$$f(x) = \frac{1}{1 + e^{-x}}$$

$$f(x) = \tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

MULTILAYER PERCEPTRON

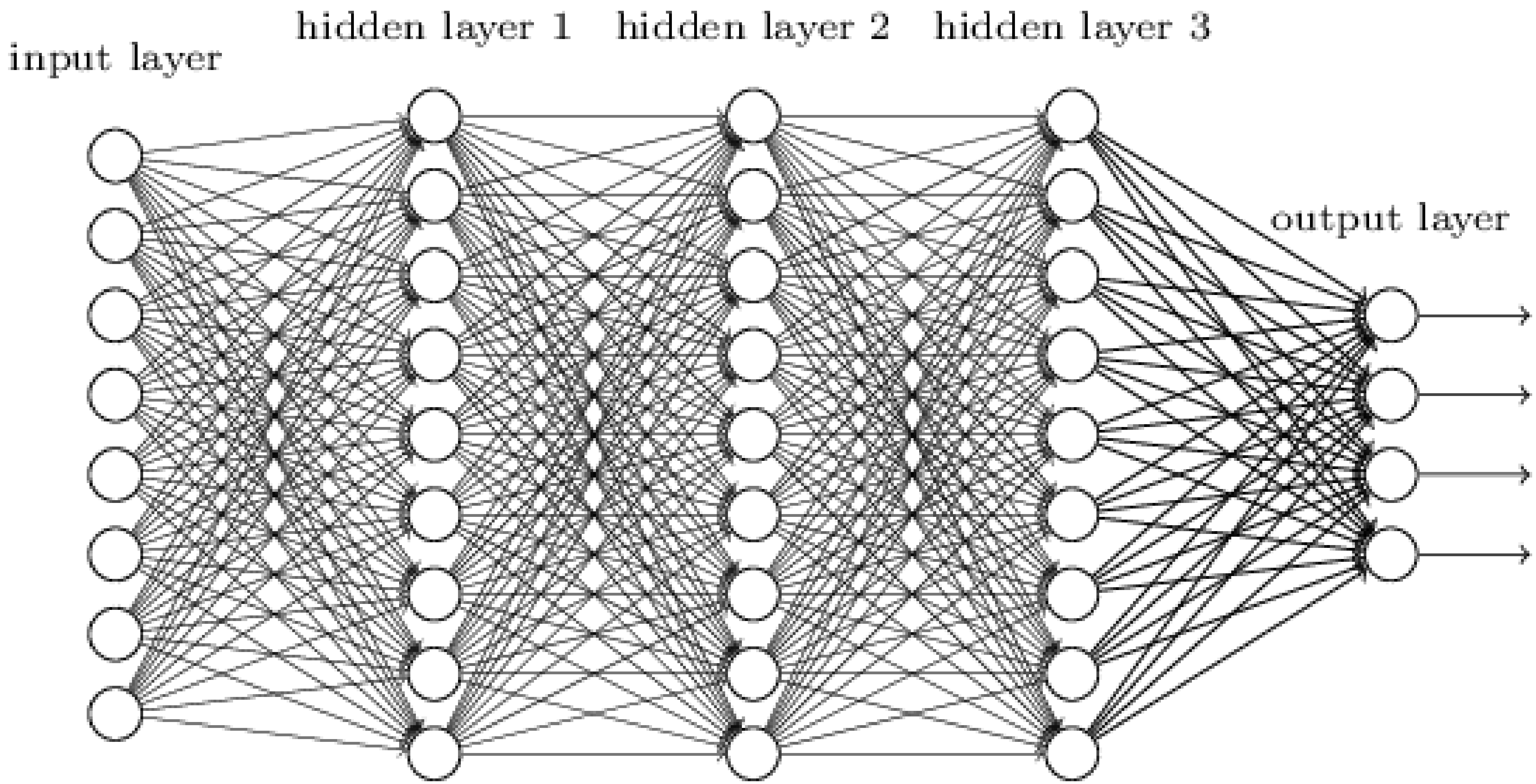
Also called a **fully connected feed-forward neural network**



Input Layer Hidden Layer Output Layer

MULTILAYER PERCEPTRON

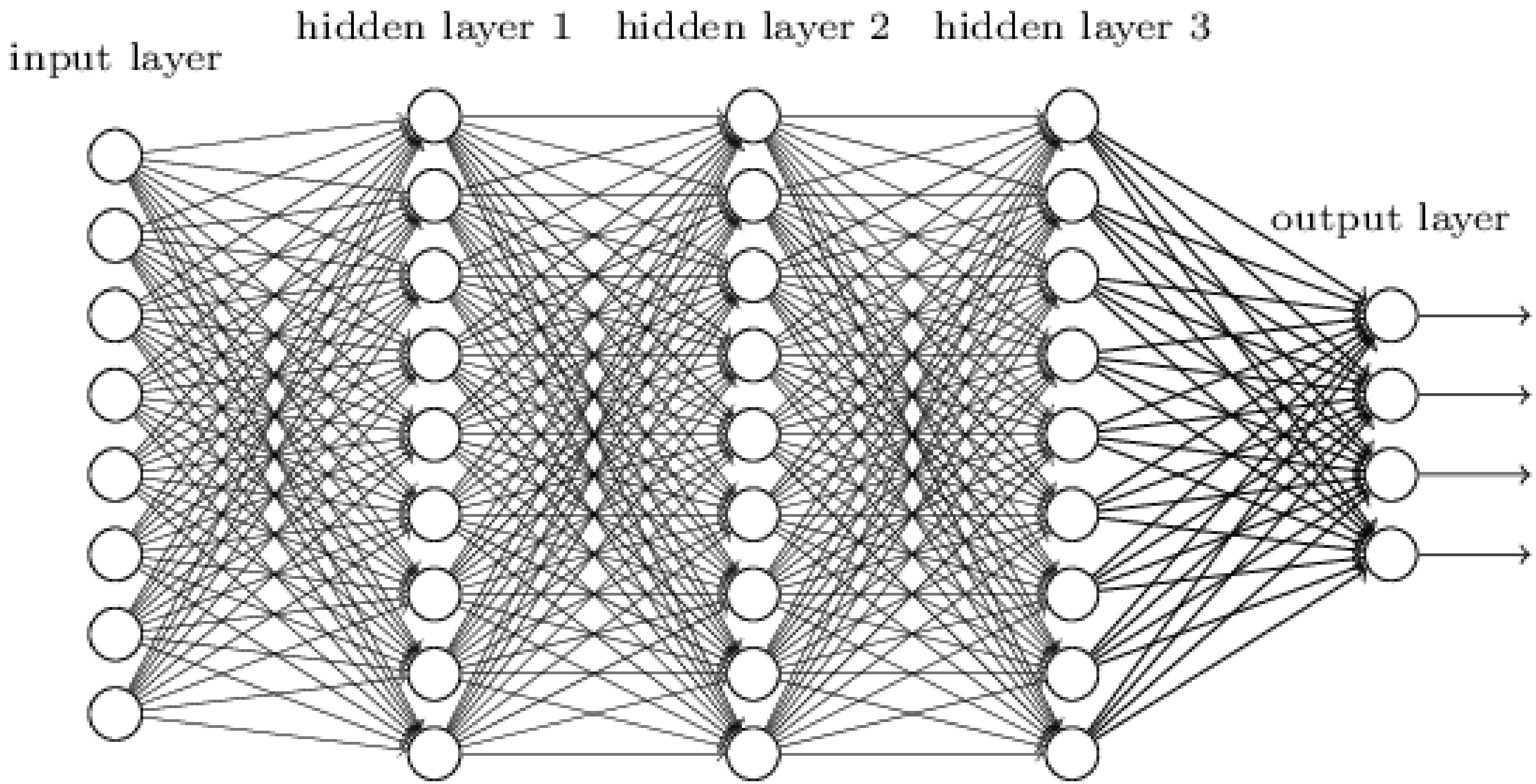
Also called a **fully connected feed-forward neural network**



PROBLEM

- How can we efficiently train the multilayer perceptron?
    - Adjust the weights at every level
    - In a computationally feasible way

MULTILAYER PERCEPTRON



Approximately, how many parameters are estimated?

NEW ANN EXCITEMENT

- In 1986 a paper was published show how to use an algorithm called "back-propagation" to train ANNs
  - The algorithm (or close relatives) was actually discovered several times prior (independently), but did not gain in popularity until 1986

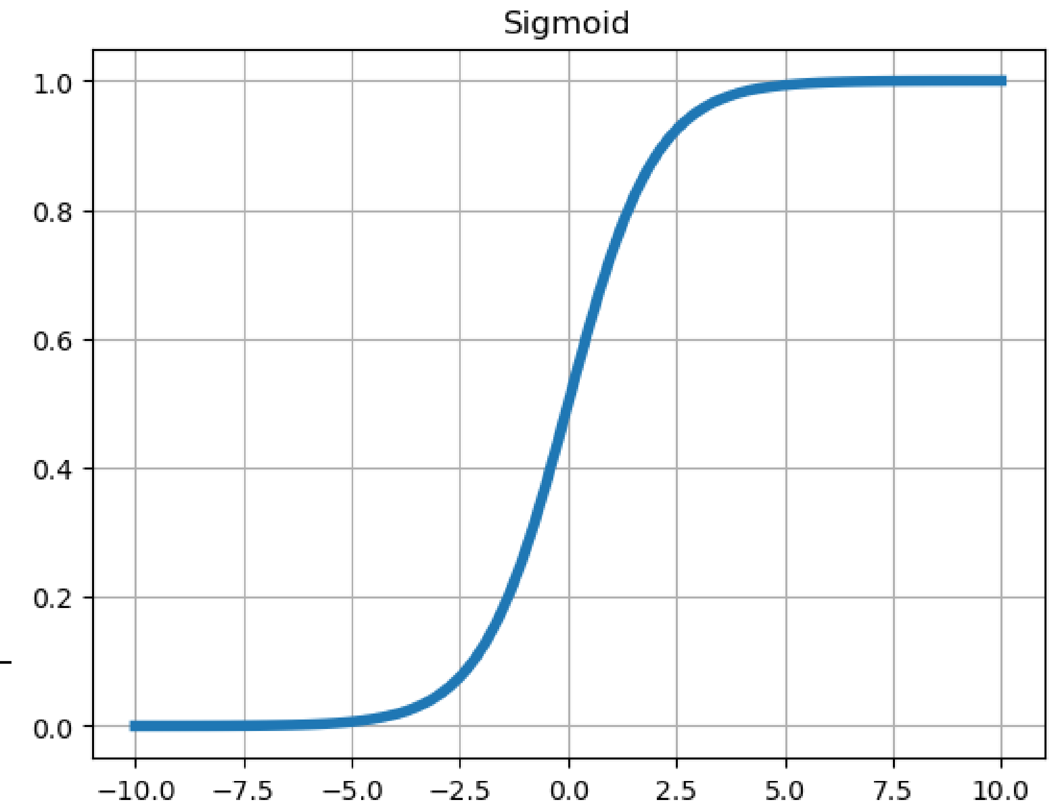
NEW ANN EXCITEMENT

- Backpropagation is a computationally efficient method for computing the gradient of a loss function
    - The gradient of the loss function for each weight is computed **using the chain rule one layer at a time**
    - It iterates backward from the last layer **to avoid redundant calculations** of intermediate terms in the chain rule
- Then weights are adjusted using gradient descent or similar

NEW ACTIVATION FUNCTION

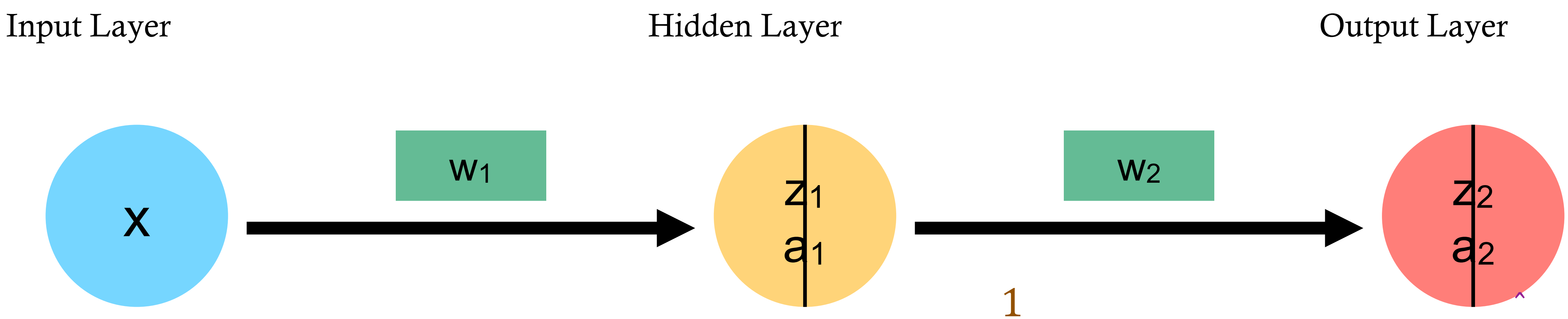
- In order for the partial derivatives to exist at every layer
  - The non-differentiable activation function (step) is changed for a differentiable activation function
  - The sigmoid (logistic) function was used in the original MLP

$$f(x) = \frac{1}{1 + e^{-x}}$$



UNDERSTANDING BACKPROPAGATION

- In order to develop intuition about the back propagation algorithm, consider a very simple neural network:



$z_1 = xw_1$ Linear combination

$a_1 = \frac{1}{1+e^{-z_1}}$ Activation

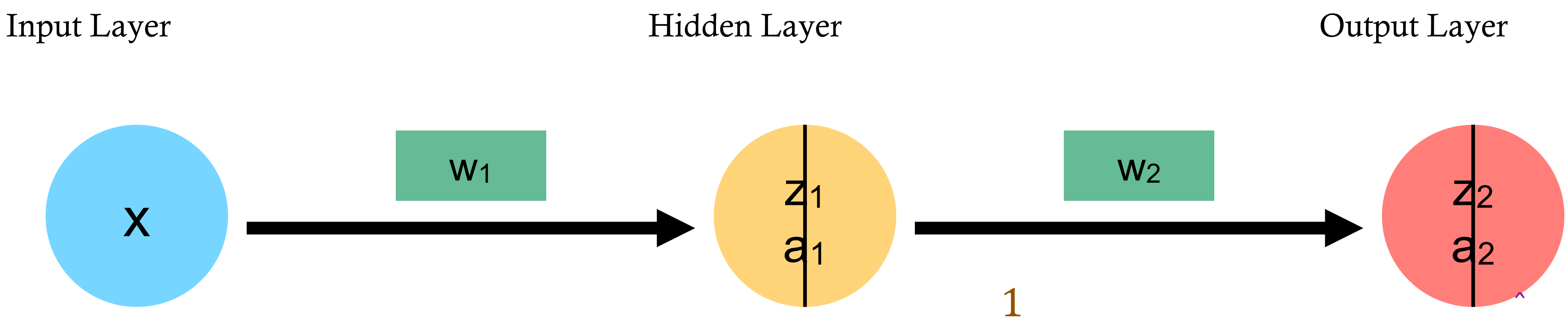
$z_2 = a_1 w_2$ Linear combination

$\hat{y} = \frac{1}{1+e^{-z_2}}$ Activation

$$J(w) = \frac{1}{2}(y - \hat{y})^2$$

UNDERSTANDING BACKPROPAGATION

- In order to develop intuition about the back propagation algorithm, consider a very simple neural network:



$z_1 = xw_1$

Linear combination

$a_1 = \frac{1}{1+e^{-z_1}}$

Activation

$z_2 = a_1 w_2$

Linear combination

$\hat{y} = \frac{1}{1+e^{-z_2}}$

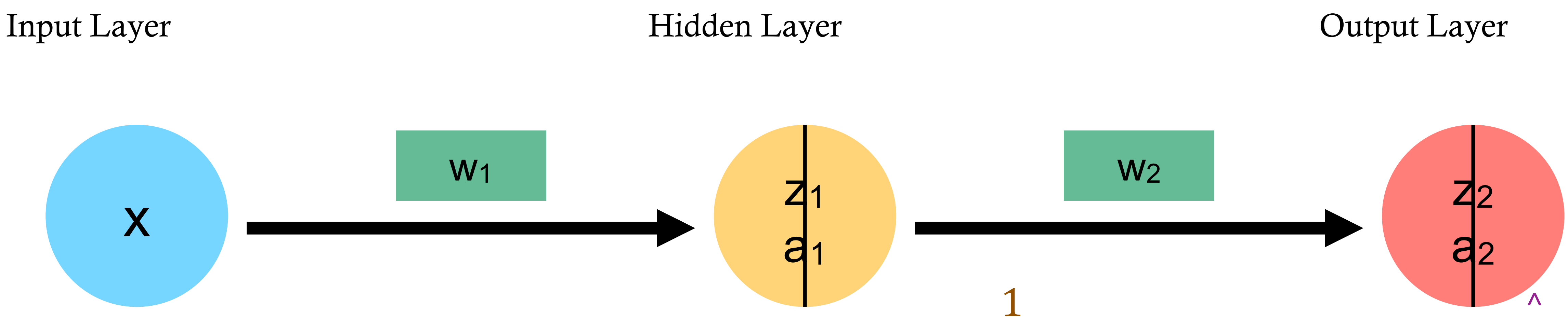
Activation

The goal is to compute the gradient of cost function *J* with respect to w1 and w2 in order find the direction of steepest descent

$$J(w) = \frac{1}{2}(y - \hat{y})^2$$

UNDERSTANDING BACKPROPAGATION

- In order to develop intuition about the back propagation algorithm, consider a very simple neural network:



$z_1 = xw_1$ Linear combination

$a_1 = \frac{1}{1+e^{-z_1}}$ Activation

$z_2 = a_1 w_2$ Linear combination

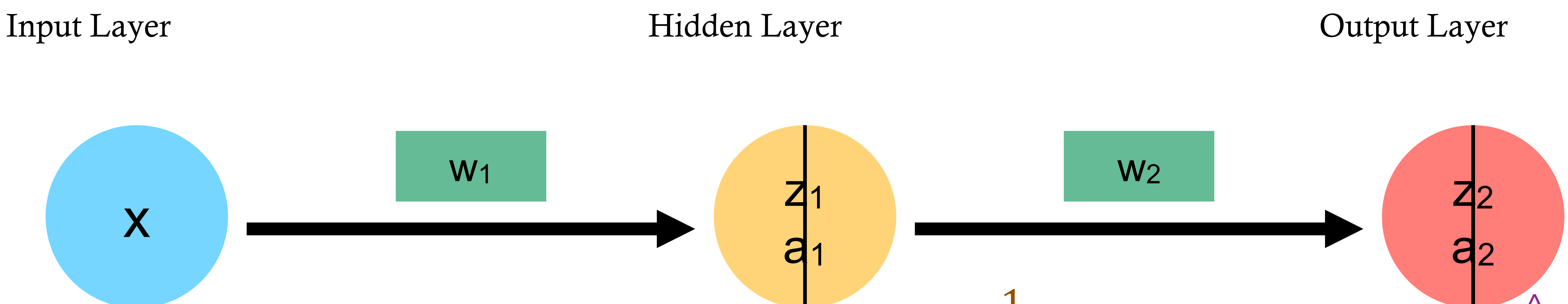
$\hat{y} = \frac{1}{1+e^{-z_2}}$ Activation

$$\frac{\partial J}{\partial w_2} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial w_2}$$

$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial a_1} \frac{\partial a_1}{\partial z_1} \frac{\partial z_1}{\partial w_1}$$

UNDERSTANDING BACKPROPAGATION

- In order to develop intuition about the back propagation algorithm, consider a very simple neural network:



$z_1 = xw_1$ Linear combination

$a_1 = \frac{1}{1+e^{-z_1}}$ Activation

$z_2 = a_1 w_2$ Linear combination

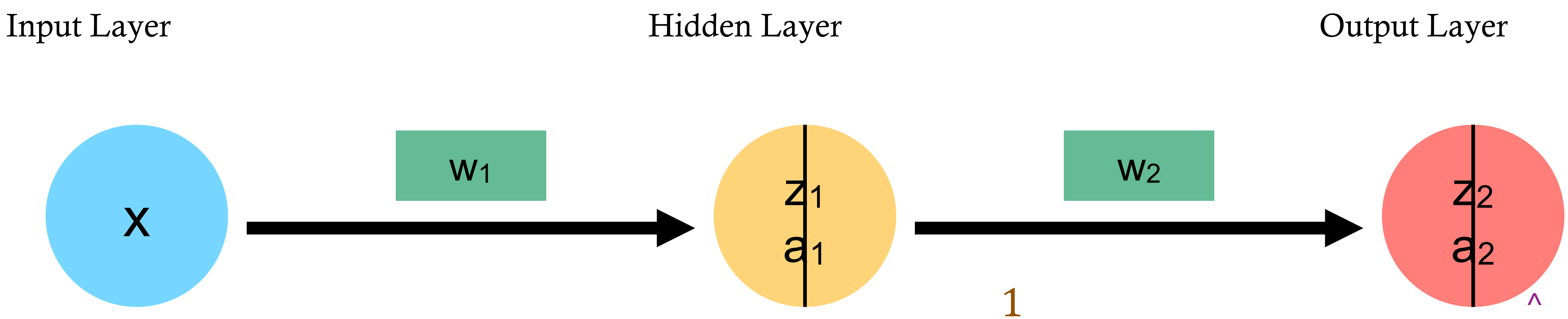
$\hat{y} = \frac{1}{1+e^{-z_2}}$ Activation

$$\frac{\partial J}{\partial w_2} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial w_2}$$

$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial a_1} \frac{\partial a_1}{\partial z_1} \frac{\partial z_1}{\partial w_2}$$

UNDERSTANDING BACKPROPAGATION

- In order to develop intuition about the back propagation algorithm, consider a very simple neural network:



$z_1 = xw_1$ Linear combination

$a_1 = \frac{1}{1+e^{-z_1}}$ Activation

$z_2 = a_1 w_2$ Linear combination

$\hat{y} = \frac{1}{1+e^{-z_2}}$ Activation

$$\frac{\partial J}{\partial w_2} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial w_2} = (y - \hat{y})(1 - \hat{y})(\hat{y})(a_1)$$

$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial a_1} \frac{\partial a_1}{\partial z_1} \frac{\partial z_1}{\partial w_2} = (y - \hat{y})(1 - \hat{y})(\hat{y}) w_2 (1 - a_1)(a_1) x$$

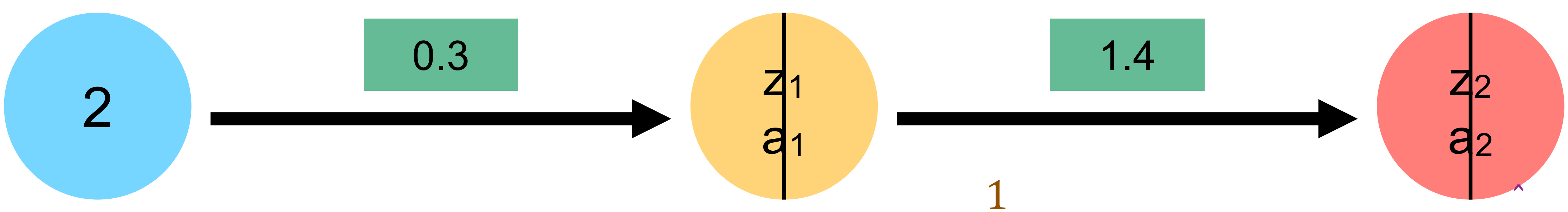
UNDERSTANDING BACKPROPAGATION

- Forward pass

Input Layer

Hidden Layer

Output Layer



$y = 5$

$z_1 = xw_1$

Linear combination

$a_1 = \frac{1}{1 + e^{-z_1}}$

Activation

$z_2 = a_1 w_2$

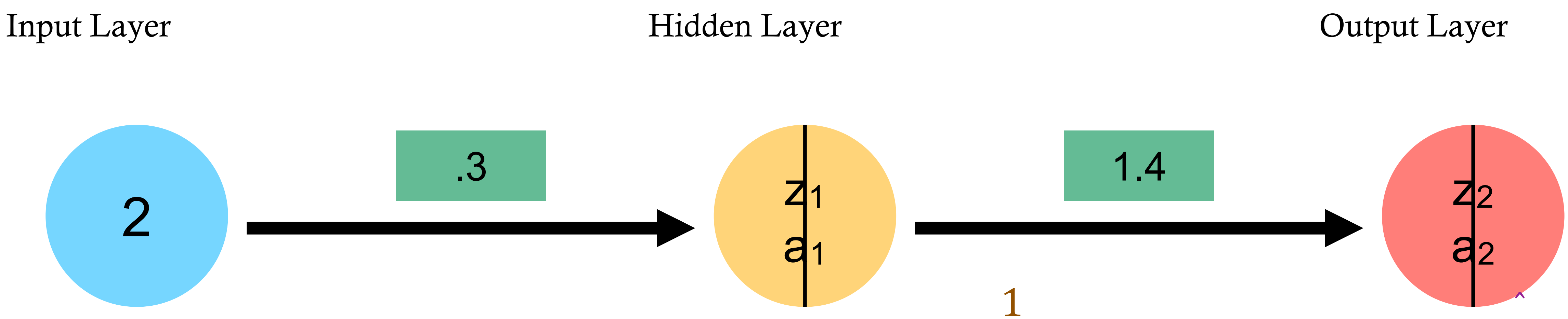
Linear combination

$\hat{y} = \frac{1}{1 + e^{-z_2}}$

Activation

UNDERSTANDING BACKPROPAGATION

- Forward pass



$y = 5$

$z_1 = x w_1$ $\qquad a_1 = \frac{1}{1 + e^{-z_1}}$

$z_2 = a_1 w_2$ $\qquad \hat{y} = \frac{1}{1 + e^{-z_2}}$

$z_1 = (.3)(2) = .6$

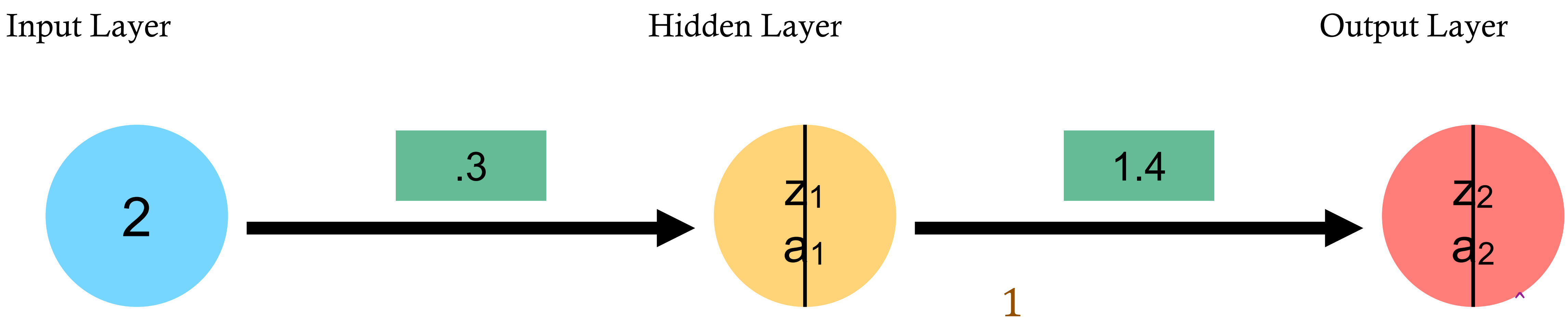
$z_2 = (.646)(1.4) = .904$

$$a_1 = \frac{1}{1 + e^{-.6}} = .646$$

$$\hat{y} = \frac{1}{1 + e^{-.904}} = .712$$

UNDERSTANDING BACKPROPAGATION

- Back propagation



$y = 5$

$z_1 = xw_1$

$a_1 = \frac{1}{1+e^{-z_1}}$

$z_2 = a_1 w_2$

$\hat{y} = \frac{1}{1+e^{-z_2}}$

$z_1 = (.3)(2) = .6$

$z_2 = (.646)(1.4) = .904$

$a_1 = \frac{1}{1+e^{-.6}} = .646$

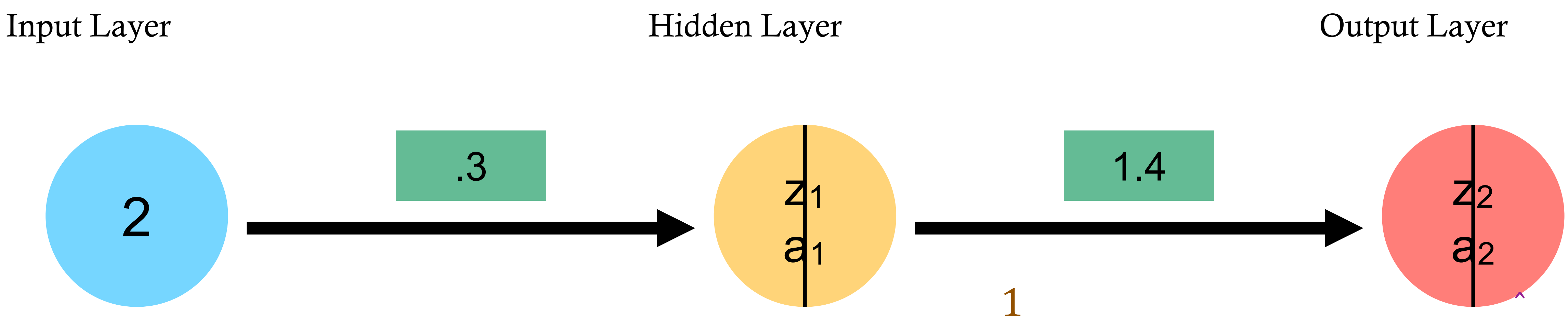
$\hat{y} = \frac{1}{1+e^{-.904}} = .712$

$$\frac{\partial J}{\partial w_2} = (y - \hat{y})(1-\hat{y})(\hat{y})(a_1)$$

$$= (5 - 0.712)(1 - 0.712)(0.712)(.646) = .568$$

UNDERSTANDING BACKPROPAGATION

- Back propagation



$y = 5$

$z_1 = xw_1$

$a_1 = \dfrac{1}{1+e^{-z_1}}$

$z_2 = a_1 w_2$

$\hat{y} = \dfrac{1}{1+e^{-z_2}}$

$z_1 = (.3)(2) = .6$

$z_2 = (.646)(1.4) = .904$

$a_1 = \dfrac{1}{1+e^{-.6}} = .646$

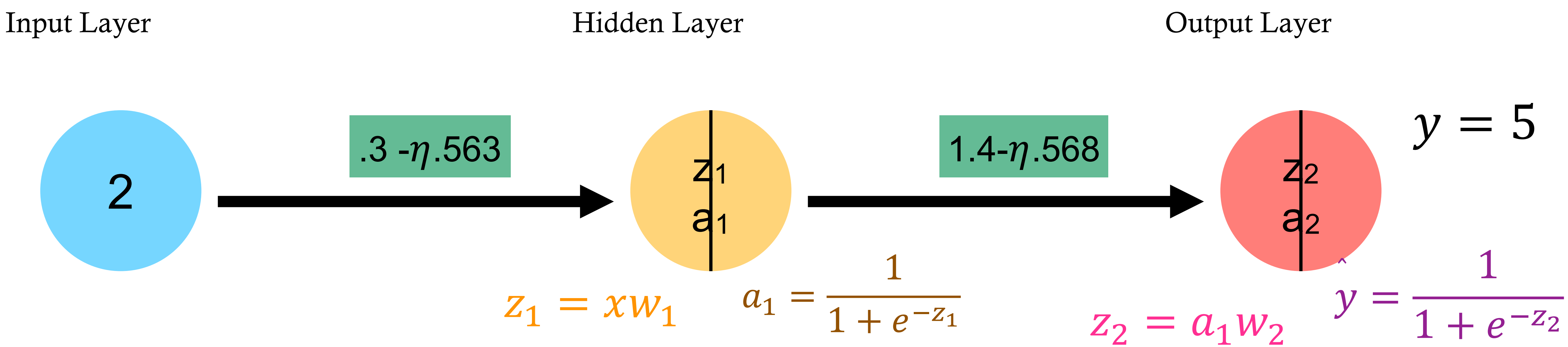
$\hat{y} = \dfrac{1}{1+e^{-.904}} = .712$

$$\frac{\partial J}{\partial w_1} = (y-\hat{y})(1-\hat{y})(\hat{y})w_2(1-a_1)(a_1)x$$

$$= (5-0.712)(1-0.712)(0.712)(1.4)(1-.646)(.646)(2) = .563$$

UNDERSTANDING BACKPROPAGATION

- Update and repeat



MORE PLACES TO UNDERSTAND

3Blue1Brown (YouTube Channel)

- Deep Learning
  - Chapter 1 (But what is a neural network? ~ 19min)
  - Chapter 2 (Gradient descent, how neural networks learn ~20min)
  - Chapter 3 (What is backpropagation really doing? ~ 13min)
  - Chapter 4 (Backpropagation calculus ~ 10min)

Jeremy Jordan Blog

https://www.jeremyjordan.me/neural-networks-training/

(This a thorough and easy-to-follow walkthrough of ANNs and backprop

OUTPUT LAYER

- The **number of neurons** in the output layer AND the **activation function** corresponds to the **desired output**
    - (what does the target look like?)
- ***Regression***
    - Usually one output layer, several activations functions possible (often the “identity” activation function)
    - For multivariate regression (multiple targets), one neuron is needed for each output dimension

OUTPUT LAYER

- The **number of neurons** in the output layer AND the **activation function** corresponds to the **desired output**
    - (what does the target look like?)
- ***Classification***
    - For binary classification, only one output neuron is needed using sigmoid activation function
    - For multiclass classification, one output neuron per class using the softmax activation function
        - Softmax is also possible for binary classification with two output neurons

VANISHING GRADIENT PROBLEM

- Phenomenon where the gradients of the loss function become increasingly smaller as the error propagates backward

- When the derivative of the activation function is less than one, multiplying small numbers repeatedly causes the gradient to become very small

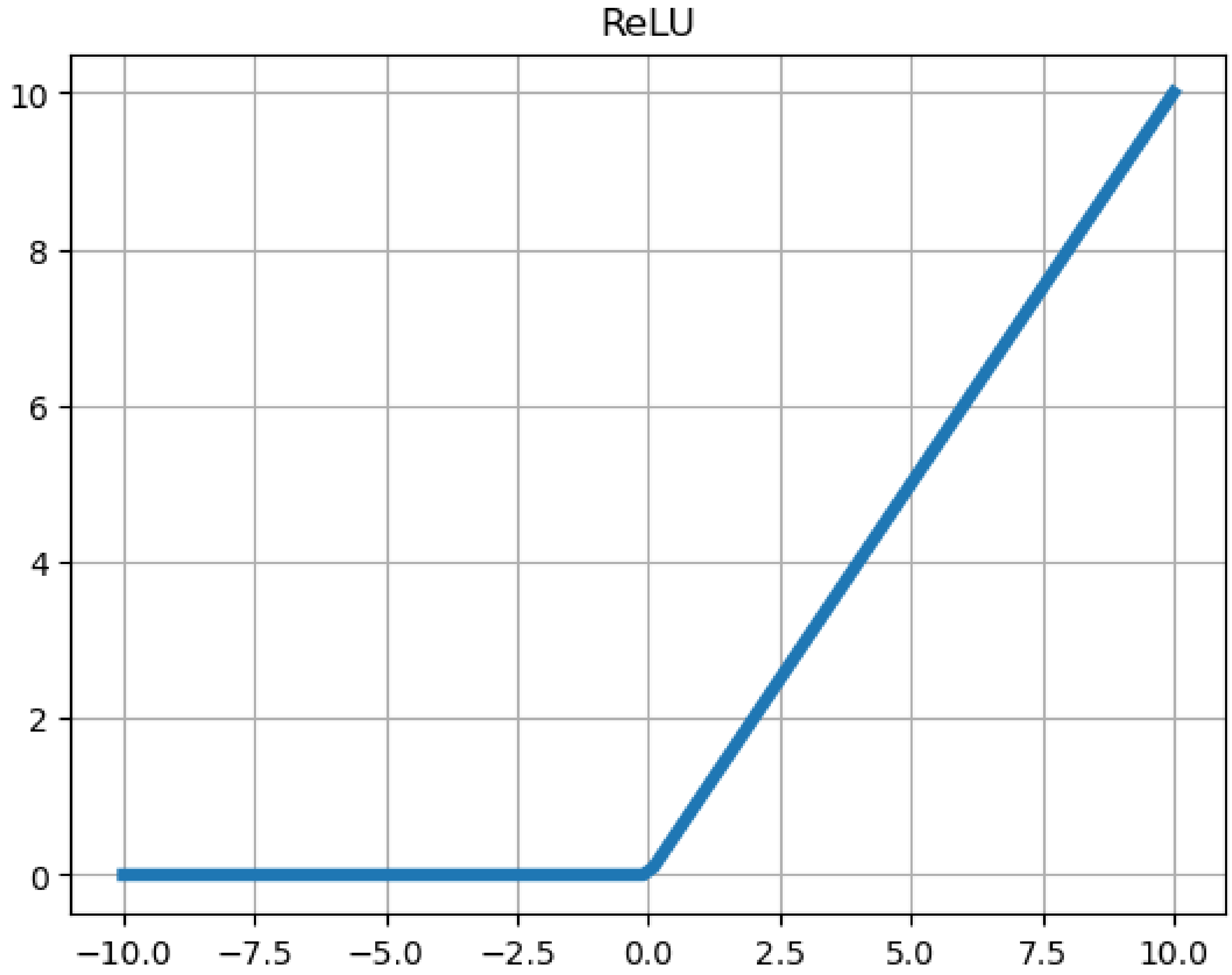
- This leads to **slow converges** and **poor performance**

- The opposite problem can also occur (exploding gradients)

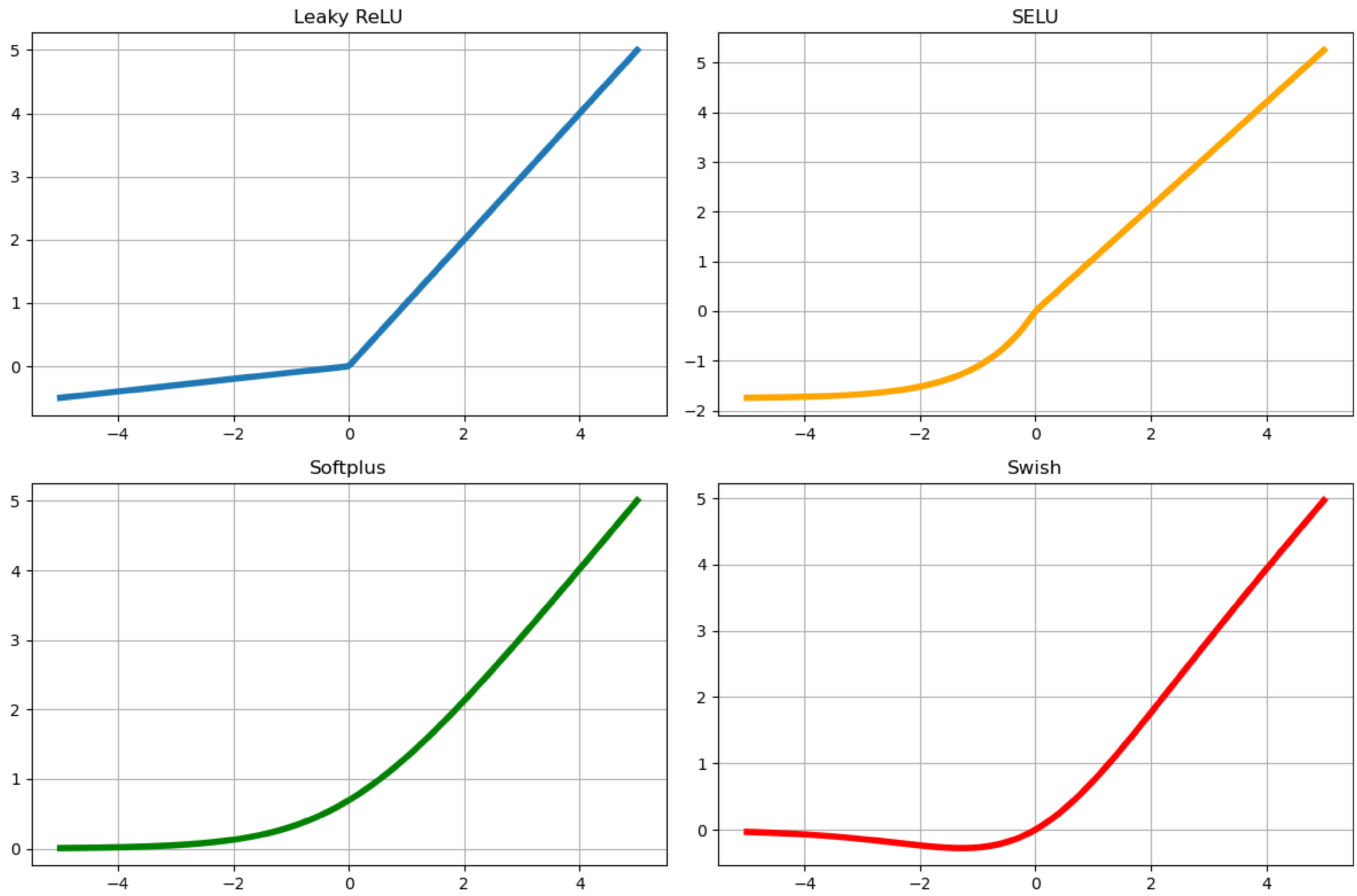
SOLUTIONS FOR UNSTABLE GRADIENTS

- Activation functions have been designed to alleviate this problem
  - ReLU and its variants
- Weight initialization techniques help in setting weights to values that reduce the risk of unstable gradients at the start
- Normalization techniques, such as batch normalization, help maintain stability by normalizing the inputs to each layer
- Some network architectures are designed to help mitigate this issue

RECTIFIED LINEAR UNIT (RELU)



RELU VARIANTS (NOT AN EXHAUSTIVE LIST)



DEEP LEARNING REVOLUTION

- Mitigating unstable gradients as well as other technological improvements led to the next (current) AI revolution
- The internet made the impossible possible
    - Crowdsourcing (e.g., Mechanical Turk) helped create massive labeled datasets (ImageNet, etc.)
- Computing power increased massively (GPUs)

- **AlexNet in 2012 demonstrated the remarkable capability of deep neural nets**

TUNING A DNN

- Architecture
  - Number of hidden layers
  - Number of neurons per hidden layer
- Other parameters
  - Learning rate
  - Optimizer
  - Batch size / epochs
  - Activation function / initialization / normalization
  - Regularization

LEARNING RATE

- The gradient descent learning rate
- Determines the size of the steps that the optimization algorithm takes
  - Too high: Might fail to converge because minimum is never found
  - Too how: Slow and can get stuck in local minima
  - Just right: Often the learning rate is adjusted over time

OPTIMIZER

- Variations of gradient descent have been developed to be faster and more stable
  - Adam: Adaptive Moment Estimation
    - Requires minimal memory
    - Computationally efficient
    - Suited for large problem (large n and/or large p)
    - Effective for noise or sparse gradients

BATCH SIZE & NUMBER OF EPOCHS

- **Batch Size:** Number of training examples used in one iteration of model training
  - Model weights are updated after computing the gradient over batches
  - Large batch sizes are processed more efficiently but can lead to instabilities (see pg 352 HOML)
- **Epoch:** One full passthrough of the entire training data
  - Each epoch consists of the number of iterations needed to use all training examples once
  - Too many epochs can lead to overfitting and too few can lead to underfitting

ACTIVATION, INITIALIZATION, NORMALIZATION

- Activation functions
  - ReLU and its variants in hidden layers
- Weight initialization
  - *He initialization* is common for ReLU activation functions
  - See pages 359-360 HOML
- Batch normalization
  - Normalize the inputs of each layer

COMMON REGULARIZATION METHODS

- **L1 and L2 regularization**
  - Add a penalty to cost function and prevent weights from becoming too large
- **Dropout**
  - At every step, every neuron has probability p of being ignored
- **Early stopping**
  - Stop training if test error begins to go up
- **Data augmentation**
  - Artificially add more data (works especially well for CNNs)

Let's take a look at the TensorFlow playground

(https://playground.tensorflow.org)